

류현식 | 신입 FE 개발자

구조 설계부터 배포까지 끝까지 책임지는 프론트엔드 개발자

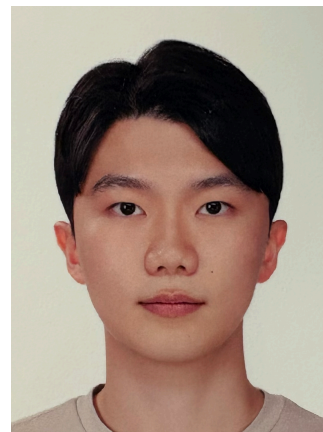
☎ Phone : 010-7316-7616

✉ Mail : rhsik2000@gmail.com

🔗 Portfolio : <https://www.hyunsik2000.com>

🐙 GitHub : <https://github.com/hyunsik2000>

📝 Blog : <https://velog.io/@hyunsik2000>



사용자가 더 안정적이고 일관된 경험으로 서비스를 이용할 수 있도록 돕는 **프론트엔드 개발자**를 목표로 하고 있습니다. 개발은 단순히 기능을 구현하는 일이 아니라, **서비스 구조를 설계하고 사용자 경험을 개선하는 과정**이라고 생각합니다.

기능 구현만큼 "왜 이 구조인가"를 먼저 고민하는 데 시간을 투자합니다. 팀장 경험을 통해 프로젝트 인증 구조, UX 흐름, 공통 컴포넌트 설계를 맡으며 팀원이 일관된 방식으로 개발할 수 있는 환경을 만들어왔습니다. 설계가 바뀌어야 할 때는 과감하게 뒤집고 재설계한 경험이 있습니다. 완성보다 올바른 방향을 더 중요하게 생각합니다.

기술 스택

Frontend	• Core : Next.js, React, TypeScript, JavaScript • Styling : Tailwind CSS • State & Data: Zustand
Infra & Deployment	• Cloud: AWS EC2, S3 • CI/CD : GitHub Actions, CodeDeploy • Deployment: Vercel
Tools	• Git, GitHub, Figma, Notion, Postman

교육 및 활동

오즈코딩스쿨 초격차캠프 프론트엔드 15기 수료
2025.09 ~ 2026.03

금오공과대학교, 컴퓨터공학과 학사 졸업
2019.03 ~ 2025.08

패스트캠퍼스 인터랙티브 웹 퍼블리싱 교육 수료
2024.11 ~ 2025.01

멋쟁이사자처럼 12기 프론트엔드
2024.03 ~ 2024.09

프로젝트

copymark - npm 라이브러리

개인 | 2026.01 ~

copymark/@lyu_danny

NPM Library - Clipboard 복사 후 Toast 알림

기술 스택

React, Vite, TypeScript

Github

<https://github.com/hyunsik2000/copymark>

- npm 패키지 배포 및 모노레포 구조 설계
라이브러리와 데모를 별도 레포로 관리하면 변경마다 재배포가 필요하였습니다. 모노레포 구조로 통합해 수정사항을 데모에 즉시 반영하였습니다.
- Lazy DOM 주입 패턴 적용
첫 복사 시점에 createRoot로 Toast를 body에 자동 삽입하여 사용자가 별도 설정 없이 clipboard 기능 사용 가능하도록 구조화하였습니다.
- 누적 다운로드 약 300회 달성
개발자가 사용할 수 있는 형태로 npm 배포를 진행 중입니다.

Ozent - 사용자 맞춤 향조 추천 서비스

OZ | 2026.02 ~ 2026.03

관리자 페이지 개발

FE 팀장 FE - 3인 / BE - 3인

기술 스택

Next.js, React, TypeScript, Tailwind CSS,
AWS S3 EC2

Web

<https://oz-scent-match.duckdns.org/>

Github

<https://github.com/team-201-created/FE>

- AWS EC2 배포
GitHub Actions + CodeDeploy로 자동 배포 파이프라인 구축 후 EC2 Ubuntu 환경에서 Nginx 리버스 프록시 구성 + PM2 무중단 배포를 자동화로 전환하여 트래픽 증가에 대비하였습니다.
- 캐시 무효화 설계
어드민 데이터에서 이전(stale)값이 노출되는 문제를 해결하기 위해 Next.js 통신 구조를 공부하였습니다. 어드민 서버 컴포넌트에 no-store 캐싱 처리와 데이터 mutation 직후엔 revalidate 호출로 이전 값이 노출되는 문제를 해결하였습니다.
- 클라이언트 인증 로직 공통화
인터셉터 패턴 토큰 주입 및 401 처리 코드를 필요한 부분에 재사용하기 위해 createFetch 함수를 만들어 공통 로직을 인터셉터에 위임하였습니다. httpOnly 쿠키 토큰은 서버에서만 읽도록 dynamic import로 분리해 클라이언트 환경 오염을 막았습니다.
- 전역 모달 시스템 설계
모달마다 개별 state로 관리하다 z-index 충돌과 중첩 모달 처리에서 한계 경험 후 Zustand Store + React Portal + 배열 스택 방식으로 통합해 DOM 최상위에서 제어하도록 설계하였습니다.
- Next 환경에서 SEO 작업
SSR 렌더링 시 Next.js metadata, sitemap, robots 설정을 통한 SEO 최적화 적용 후 Lighthouse SEO 82점 → 100점으로 개선하였습니다. 실제 서비스로서의 노출 지표 확인을 위해 Google Search Console 색인 등록 후 검색 노출 및 클릭 수 지표를 확인하였습니다.

LMS 통합 관리 시스템

OZ Externship | 2026.01 ~ 2026.02

쪽지시험 관리 페이지 개발

FE 팀장 FE - 3인 / BE - 3인

기술 스택

React, Vite, TypeScript, Tailwind CSS
Axios, Zustand

Github

https://github.com/OZ-Coding-School/oz_externship_fe_06_team1

- 전역 에러 핸들링 로직 설계
공통 Axios 인스턴스와 API 호출을 설계하여, 팀원에게 로딩 상태와 전역적 에러 처리를 자동적으로 처리할 수 있도록 표준화 후 지역적 에러는 객체 상태로 받아 커스텀 할 수 있는 구조로 개선하였습니다.
 - 초기 설계: Axios interceptor → 전역 스토어 중앙집권식 구조 적용
 - 문제 발견: 로그인 폼 등 지역 처리가 필요한 케이스에서 한계
 - 재설계: useAxios 혹은 onError 콜백이 true 반환 시 전역 전파 차단 + ErrorBoundary로 런타임 에러까지 3계층 방어
 - 개선 후 서비스 다운 이슈 0건
- 범용 데이터 테이블 컴포넌트 설계
페이지마다 반복적으로 사용되는 테이블 UI를 복합 컴포넌트 패턴으로 추상화하여 사용 가이드 문서와 함께 팀 내 공유하였습니다.
- 코드 품질 관리 자동화
Husky와 lint-staged를 도입하여 커밋 시 자동 린트 검사 프로세스를 구축하여 리뷰 과정에서 코드와 컨벤션 스타일 관련 피드백을 줄여, 코드 리뷰에 집중할 수 있는 환경을 구축하였습니다.

Fizz! - Wellness 챌린지 숏폼 플랫폼

멋쟁이 사자처럼 | 2024.07 ~ 2024.08

영상 스트리밍 및 업로드 전담 개발

FE 팀장 FE - 3인 / BE - 3인

기술 스택

React, JavaScript, Tailwind CSS, SSE

Github

https://github.com/Fizz-challenge/Fizz_front

- hls.js 기반 스트리밍 최적화
영상을 다룰 때 일반 <video> 태그로는 m3u8 형식의 청크 분할 스트리밍이 불가능해 hls.js 도입 후 버퍼 크기 조정하여 재생까지의 지연 시간을 단축하였습니다.
- 메모리 누수 방지 및 성능 관리
쇼츠 특성상 사용자가 영상을 빠르게 넘기며 hls 인스턴스가 누적되는 문제 발생하여 Intersection Observer API와 결합해 화면에 보이는 영상에만 hls 인스턴스를 활성화하고, 벗어난 영상은 즉시 리소스 해제를 시켜 장시간 스크롤 시 메모리 누적을 방어하였습니다.
- 영상 업로드 구조 설계
서버를 거쳐 업로드 시 병목이 발생하는 구조를 개선하기 위해 BE팀에 스토리지 업로드의 필요성을 제안 하였습니다. 서버로 영상 업로드하던 과정을 AWS S3에 영상 파일 업로드하여 관리하는 형태로 변경하여 영상 업로드 과정 중 속도를 향상시키고 안정성을 지킬 수 있었습니다.

CaféFlow - 실시간 카페 좌석 현황 서비스

학과 졸업작품 | 2024.03 ~ 2024.06

Map 및 실시간 통신 구조 개발

FE 팀장 FE - 3인 / BE - 3인

기술 스택

React, JavaScript, Naver Map API, SocketJS

Github

<https://github.com/Cafe-Flow/CafeFlow-FrontEnd>

- 실시간 혼잡도 시각화 - 현재 카페 내부에 사용 가능한 좌석 비율을 값으로 받음
REST 폴링으로는 혼잡도 변경을 즉시 반영하기 어렵고 불필요한 요청이 반복되는 문제가 발생하였습니다. 페이지 진입 시 STOMP + SocketJS로 클라이언트별 Subscribe 객체 연결을 유지하여 혼잡도 데이터를 실시간으로 수신 받아 사용자에게 잘 보일 수 있는 마커 색상으로 업데이트하였습니다.
- 1:1 채팅 기능 구현
사용자가 채팅을 요청할 때만 채팅 전용 STOMP 클라이언트 객체를 생성해 Subscribe/Publish 연결. 카페 관리자-사용자 간 실시간 채팅을 지원하였습니다.
- 전역 상태 설계 및 재사용 컴포넌트
사용자 정보를 여러 컴포넌트에서 props로 반복 전달하는 drilling 문제를 Context API로 중앙화 시켜 중복되는 로직을 줄였으며 커뮤니티 게시판 테이블도 재사용 컴포넌트로 추상화해 중복 구현 제거하였습니다.

자격

공인 TOEIC 725

2025.02

정보처리기사

2024.12